



SocioMart

Community Food Marketplace — Project Report & Setup Guide

Motive. Every neighbourhood has talented home cooks whose food never reaches neighbours beyond their own floor. SocioMart connects home kitchens with people in the same society: cooks publish today's menu, neighbours browse a live marketplace, add dishes to a cart and place orders — while sellers manage kitchens, offerings, prices and order status from a simple dashboard. It is built as a real production-style product: secure OTP login, a clean REST API, persistent storage, Swagger docs and a mobile-first single-page app.

PROJECT

SocioMart `my-first-spring-api`

REPOSITORY

github.com/utkarshnikhare/my-first-spring-api

STACK

Java 21 · Spring Boot 4.1.1 · H2 · JS SPA

VERSION

v1.0 — MVP complete

STATUS

Core flows done & E2E-tested

DATE

31 August 2026

1. Project Overview

SocioMart is a full-stack **community food marketplace**: home cooks ("sellers") run a digital kitchen inside their housing society; neighbours ("buyers") discover dishes, order online, pay, track the order to delivery and rate it — the complete hyperlocal food-ordering loop in one product.

Why it exists

- **For home cooks:** no shop or tech skills needed — OTP login, create a kitchen, post today's dishes, take orders from a dashboard.
- **For neighbours:** one place to see every home kitchen in the society, browse, cart and order in a few taps.
- **For the community:** extra income for cooks, fresh homemade food for residents.
- **As a learning product:** deliberately built to production standards — layered Spring Boot backend, security, validation, real persistence, documented REST API, polished SPA.

What the finished product does today

Buyer journey (100% click-driven)	Seller journey (100% click-driven)
1. Mobile number → OTP (shown on screen in demo mode) → logged in 2. Browse marketplace: kitchen & dish cards, search, category chips 3. Tap a dish → quantity modal → Add to Order 4. Floating cart bar → cart → Checkout → delivery details → Place Order 5. Payment screen → I HAVE PAID → track order 6. Orders tab: history, detail, cancel (PENDING), reorder, rate ★	1. Profile → Start Selling (one-tap upgrade) 2. Create kitchen: name, cuisine, description, image 3. Sell dashboard: Offerings / Orders / Kitchen tabs 4. Add dishes with price, description, category 5. Incoming orders live → confirm → ready → delivered 6. Edit prices/availability; pause or delete dishes

2. What Has Been Used — Technology Stack

Layer	Technology	Why it is used
Language	Java 21 (LTS)	Modern LTS runtime; records, switch expressions
Framework	Spring Boot 4.1.1 (starter-parent)	Auto-configuration, embedded Tomcat, industry standard
Web / REST	spring-boot-starter-webmvc	REST controllers, JSON, static content serving
Security	spring-boot-starter-security (Spring Security 7)	OTP session auth, per-role endpoint protection, CSRF
Validation	spring-boot-starter-validation	Declarative bean validation → clean 400 errors
Persistence	spring-boot-starter-data-jpa + Hibernate	Repositories, entity mapping, schema auto-update
Database	H2 in file mode jdbc:h2:file:./data/sociomartdb	Zero-install persistence; survives restarts; console at /h2-console
API docs	springdoc-openapi 3.1.0	Live Swagger UI at /swagger-ui.html
Frontend	Vanilla HTML5 / CSS3 / ES6 SPA (no framework)	Zero build step; hash router; mobile-first UI
Build	Maven via included wrapper (mvnw)	Reproducible builds; no Maven install needed
Testing	28-step scripted E2E run (API + UI click flows)	All core flows verified end-to-end

Key config: app port 8081 · 30-min sessions · JPA ddl-auto=update · no external services, no environment variables required.

3. Architecture & What Was Built

3.1 Layered architecture & project structure

```
my-first-spring-api/           (repo root)
└─ my-first-spring-api/       (Maven project)
  ├── mvnw / mvnw.cmd         Maven wrapper — no Maven install needed
  ├── pom.xml                 Spring Boot 4.1.1 · Java 21 · dependencies
  ├── data/sociomartdb.mv.db   H2 file database (created on first run)
  └─ src/main/
    ├── java/com/example/my_first_spring_api/
    │   ├── MyFirstSpringApiApplication.java (main entry point)
    │   ├── SecurityConfig.java             (route security rules)
    │   ├── GlobalExceptionHandler.java     (consistent JSON ApiErrorDto)
    │   ├── controller/ Auth · Marketplace · Seller · BuyerOrder · BuyerProfile
    │   ├── service/ Buyer · Kitchen · Marketplace · Order · Otp · Seller
    │   ├── repository/ Spring Data JPA interfaces (6 repositories)
    │   ├── model/ 9 JPA entities (User, Kitchen, Product, Order...)
    │   ├── dto/ request/response payloads with validation
    │   └─ exception/ domain exceptions → mapped to HTTP status codes
    └─ resources/
        ├── application.properties (port 8081, H2 file DB, Swagger)
        └─ static/ index.html · css/styles.css · js/app.js (the SPA)
```

3.2 REST API (secure, JSON, documented in Swagger)

Area	Representative endpoints	Purpose
Auth /api/auth	POST /otp/request · POST /otp/verify · GET /me · POST /become-seller · POST /logout	OTP login, session, role upgrade
Marketplace (public)	GET /api/marketplace · GET /api/kitchens · GET /api/kitchens/{name}/products · GET /api/products/{id}	Browse kitchens, dishes, search
Buyer orders /api/buyer/orders	POST GET DELETE /draft · POST /place · GET /my · GET /{id} · PUT .../payment · POST .../rating · POST .../reorder	Cart, checkout, payment ★, history, reorder
Buyer profile	GET/PUT /api/buyer/profile	Name / flat / phone
Seller /api/seller	GET POST /kitchen · PUT /kitchen/{id} · GET POST /products · PUT DELETE /products/{id} · GET /orders · PATCH /orders/{id}/status	Kitchen & menu management, order lifecycle

3.3 Data model (6 core entities, 9 tables)

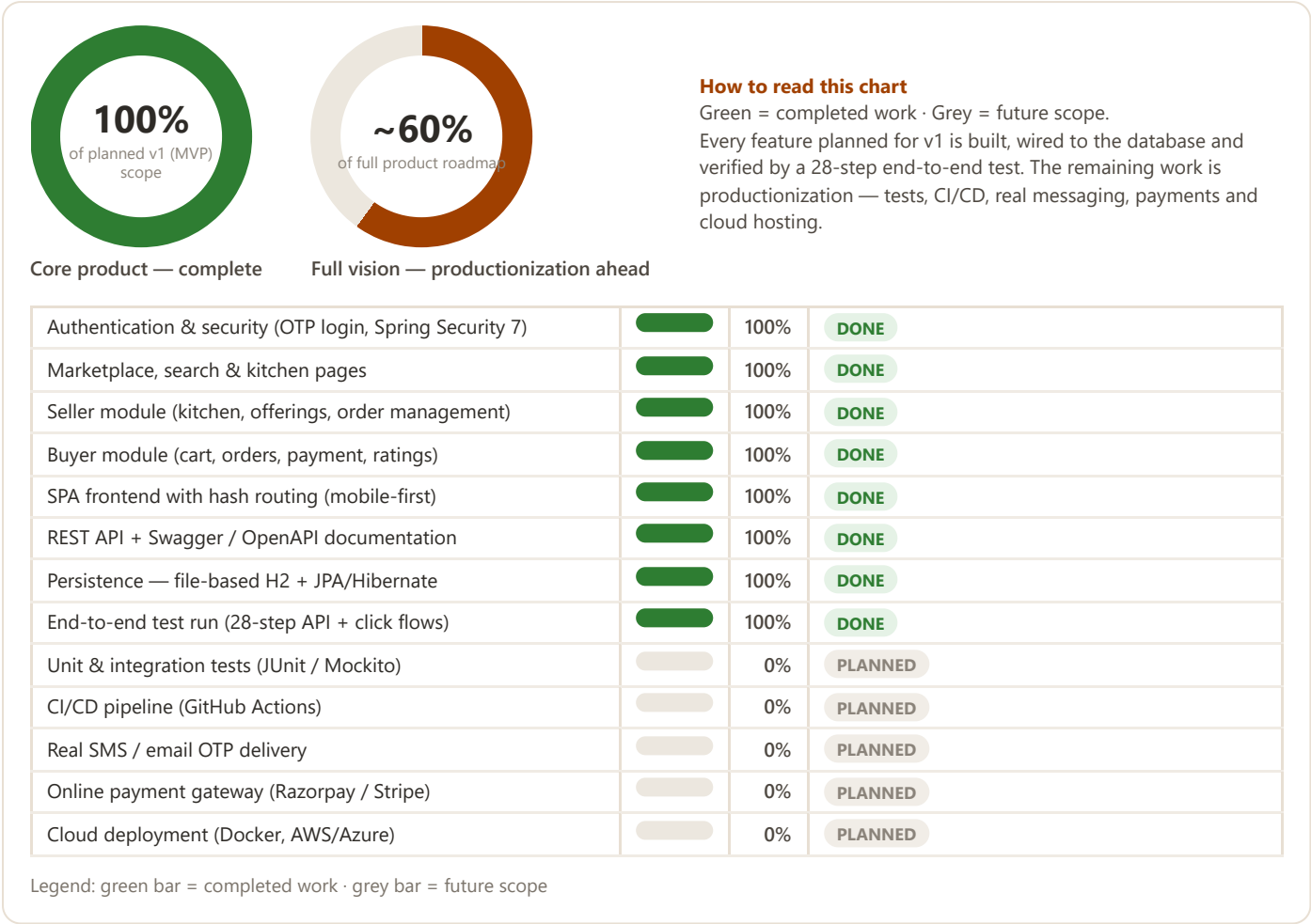
Entity	Key fields & relationships
User	mobileNumber (unique), name, flatHouseNumber, role (USER SELLER)
OtpVerification	mobileNumber, otpCode, expiresAt, verified — short-lived login codes
Kitchen	name (URL slug), displayName, cuisine, description, imageUrl, open · seller → User
Product	name, description, price, category, imageUrl, available · kitchen → Kitchen
Order	status PENDING → CONFIRMED → READY → DELIVERED / CANCELLED · paymentStatus PENDING → PAID · total, delivery fields, rating
OrderItem	order, product, quantity & price snapshots — cart and history lines

Business rules: a cart (draft order) is locked to the first kitchen added from — switching kitchens clears it; only unpaid PENDING orders can be cancelled; sellers only see/modify their own kitchen's data; every error returns a consistent JSON ApiErrorDto .

3.4 Frontend SPA

- Served by Spring Boot itself — **no Node.js, no build step:** clone, run, open browser.
- Hash routing (#/home , #/kitchen/:slug , #/my-orders , #/checkout , #/payment/:id , #/orders , #/sell , #/profile) with bottom tab bar, floating cart bar, auth-aware inline OTP prompts.

4. Project Progress Chart — Completed vs Future Scope



5. Future Scope (roadmap)

- P1

Quality: JUnit/Mockito unit & integration tests; GitHub Actions CI (build + test on every push)
- P1

Data: PostgreSQL profile for production; Flyway DB migrations
- P2

Real OTP: SMS provider (Twilio / MSG91) to replace demo OTP
- P2

Payments: Razorpay/Stripe gateway with webhooks (UPI/cards) + refunds
- P2

Notifications & packaging: order-status emails / WhatsApp; Dockerfile for easy deploys
- P3

Growth: cloud hosting, admin analytics, delivery tracking, photo reviews, PWA/mobile apps
- All v1 (MVP) features complete and E2E-tested · productionization items planned as listed above

6. Sharing via GitHub — Owner Steps

Status (done): the complete SocioMart project is already committed & pushed to `main` on GitHub — including the Spring Boot 4 compatibility fixes for Swagger and the H2 console (`pom.xml`). Local-only files (H2 database, logs, test scripts, build output) are excluded by `.gitignore` , so your friend's clone contains only clean project files + this PDF.

Whenever you change code, share it with 3 commands (run in the repo root):

```
git add -A
git commit -m "Describe what you changed"
git push origin HEAD:main
```

Then send your friend the repo URL: `https://github.com/utkarshnikhare/my-first-spring-api` (public — no access setup needed) — plus this PDF.

7. Complete Setup Guide for Your Friend

Step 1 — Install the software (one-time, ~15 minutes)

Tool	What to get	Notes / verify
1. JDK 21	Eclipse Temurin 21 (LTS) — <code>adoptium.net</code>	Windows <code>.msi</code> → tick "Set JAVA_HOME". Verify: <code>java -version</code> shows <code>21.x</code>
2. VS Code	<code>code.visualstudio.com</code>	Any OS (Windows/macOS/Linux)
3. Git	<code>git-scm.com</code>	Verify: <code>git --version</code>
4. VS Code extensions	<i>Extension Pack for Java + Spring Boot</i> <i>Extension Pack</i> (VMware)	Install from the Extensions panel (<code>Ctrl+Shift+X</code>) — adds Java debugger, Maven support, Spring Boot dashboard
NOT needed	Maven · Node.js · MySQL/Postgres · Docker	The Maven <i>wrapper</i> (<code>mvnw</code>) downloads Maven itself; H2 database is a bundled dependency; the SPA needs no build tools

Step 2 — Clone the repo & open it in VS Code

```
git clone https://github.com/utkarshnikhare/my-first-spring-api.git
cd my-first-spring-api
code .
```

- Wait ~1 minute while the Java extensions import the project and download dependencies (watch the blue status bar; trust the authors if prompted).
- The Maven project is the inner `my-first-spring-api` folder (contains `pom.xml`) — open/refresh it if VS Code shows two roots.

Step 3 — Run the app

Option	How
Easiest — VS Code	Open <code>MyFirstSpringApiApplication.java</code> → press F5 → choose <i>Java</i> if asked. Or use the Spring Boot Dashboard sidebar → right-click the app → <i>Start</i> .
Terminal — Windows	In VS Code terminal (PowerShell), from the inner <code>my-first-spring-api</code> folder: <code>.\mvnw.cmd spring-boot:run</code>
Terminal — macOS/Linux	<code>./mvnw spring-boot:run</code>

Wait for `Started MyFirstSpringApiApplication` in the console. The **first run downloads dependencies (2–5 min)**; later runs are fast. The database file `data/sociomartdb.mv.db` is created automatically.

Step 4 — Open the app & verify it works

URL	What it is
<code>http://localhost:8081</code>	The SocioMart app — boots on #/home with an empty marketplace
<code>http://localhost:8081/swagger-ui.html</code>	Live API documentation (Swagger UI) · JSON at <code>/v3/api-docs</code>
<code>http://localhost:8081/h2-console</code>	DB console — JDBC URL <code>jdbc:h2:file:./data/sociomartdb</code> , user <code>sa</code> , blank password

Try the full demo (5 minutes): register with any 10-digit mobile number → the OTP is displayed right in the dialog (demo mode — no SMS setup needed) → log in → Profile → **Start Selling** → create a kitchen → add 2–3 dishes → in a second browser/incognito window register as a different buyer → add dishes to the cart → checkout → place order → **I HAVE PAID** → back in the seller window, confirm → ready → deliver the order. That exercises every core feature.

Stop the app: `Ctrl+C` in the terminal or *Stop* in VS Code. **Factory reset:** stop the app and delete the `data/` folder — it is recreated empty on next start.

Troubleshooting

Problem	Fix
<code>java -version</code> shows 17/11/8 or error	JDK 21 not installed or <code>JAVA_HOME</code> not set — reinstall Temurin 21 with “Set <code>JAVA_HOME</code> ” ticked, restart VS Code
Port 8081 already in use	Another instance is running — stop it, or change <code>server.port</code> in <code>application.properties</code>
<code>mvnw</code> “script execution disabled” (PowerShell)	Use <code>.\mvnw.cmd spring-boot:run</code> (the <code>.cmd</code> version), or run <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> once
First build is slow / downloads fail	Normal on first run (2–5 min); check internet connection, then re-run — Maven caches downloads
VS Code shows “Language Support for Java” loading forever	Run Java: Clean Java Language Server Workspace from the Command Palette (<code>Ctrl+Shift+P</code>) and reload
Marketplace is empty	Expected on a fresh database — create a seller kitchen first (see demo above)

Cheat sheet

```
git clone https://github.com/utkarshnikhare/my-first-spring-api.git # get the code
cd my-first-spring-api; code . # open in VS Code
cd my-first-spring-api; .\mvnw.cmd spring-boot:run # run (Windows)
# macOS/Linux: ./mvnw spring-boot:run
# then open: http://localhost:8081 · /swagger-ui.html · /h2-console
```